

sample - Prompting Guide

Updated 2026-10-02 - source: SAMPLE — generic example prompts, not real data

EXCELLENT 3

GOOD 1

BAD 3

REVIEWED 7

Most common gap: E1 (success is defined) — the most frequent gap in this sample

Strongest habit: D9/D10 (decomposition fit, structural economy) — met on every applicable prompt

Trend: first checkpoint — no trend yet (a real guide shows improving/flat/regressing once you have 2+ /analyse runs)

Your Focus Right Now

E1 Success is defined (provisional - still building a baseline): E1 (success is defined) sits at level 0.45 across 5 observations, the lowest of any dimension with real evidence

Pace: insufficient_data

Full per-dimension plan: [progress/sample.md](#)

How to read each prompt below. The **scorecard** lists every rubric dimension with a status - **Met**, **Partial**, **Missing**, or **n/a** (not needed by this task). The score is the weighted roll-up of the applicable rows (Met = full, Partial = half, Missing = 0). The **transformation table** turns each gap into a concrete rewrite and the principle it teaches.

What excellent looks like

EXCELLENT · score 100 · **STRONG** Spec + exact output format + testable acceptance line

one_shot · 2026-08-10

"Add a `--dry-run` flag to scripts/deploy.sh that prints each command it would run, prefixed with "[dry-run]", and exits 0 without executing anything. Acceptance: `bash scripts/deploy.sh --dry-run` makes no network calls and prints one line per docker/kubectl command."

Why it lands here. Names the exact file and flag (D1/D2), pins the output shape (D3), and closes with a binary acceptance line (E1/E2) that also anticipates the main failure mode (E4).

Rubric scorecard

Rubric dimension	Status	Why (evidence)
LAYER 1 - DESIGN (does the prompt drive good behaviour?)		
D1 Clarity & explicitness	Met	led by the action verb 'Add'; one unambiguous instruction
D2 Specificity & constraints	Met	scope is scripts/deploy.sh; boundary 'without executing anything' is explicit
D3 Output format & length	Met	'[dry-run]' prefix + one line per command pins the shape
D4 Context & motivation	n/a	the why wouldn't change the implementation
D5 Grounding / reference	n/a	agent can read deploy.sh directly
D6 Examples (show-not-tell)	n/a	format fully described without an example
D7 Positive framing	Met	every instruction says what TO do
D8 Uncertainty handling	n/a	verifiable coding task, no hallucination risk
D9 Decomposition fit	Met	simple enough for one prompt

Rubric dimension	Status	Why (evidence)
D10 Structural economy	Met	no over-engineering, no dead instructions
LAYER 2 - EVALUABILITY (can success be measured?)		
E1 Success is defined	Met	the acceptance line defines correct output
E2 Criteria are measurable	Met	'makes no network calls and prints one line per ... command' is binary
E3 Multidimensional coverage	Met	covers both correctness (no execution) and format (prefix, one line each)
E4 Failure modes anticipated	Met	'exits 0 without executing anything' guards the main failure mode

No changes needed - imitate this one.

EXCELLENT · score 88 · STRONG Terse chain step, correctly so

chain_step · 2026-08-10

"now push it"

Why it lands here. The referent ('it') and action resolve cleanly from the session; brevity here is correct, not under-specification.

Chain step. Follows the dry-run-flag spec and 'make it work' in the same session — 'it' resolves to the change just worked on.

Rubric scorecard

Rubric dimension	Status	Why (evidence)
LAYER 1 - DESIGN (does the prompt drive good behaviour?)		
D1 Clarity & explicitness	Met	'it' resolves via session context; the verb (push) is unambiguous
D2 Specificity & constraints	n/a	a push has no extra constraints to state beyond the inherited branch
D3 Output format & length	n/a	no output format applies to a git push
D4 Context & motivation	n/a	motivation wouldn't change the action
D5 Grounding / reference	n/a	no external grounding needed
D6 Examples (show-not-tell)	n/a	no example needed
D7 Positive framing	n/a	too short to carry a framing signal either way
D8 Uncertainty handling	n/a	not a factual-risk task
D9 Decomposition fit	Met	a single step, not artificially split
D10 Structural economy	Met	appropriately terse for a chain step
LAYER 2 - EVALUABILITY (can success be measured?)		
E1 Success is defined	Met	success = the push completes against the current branch, inherited from context
E2 Criteria are measurable	Partial	doesn't state how the push's success would be confirmed (e.g. check the command's exit status)
E3 Multidimensional coverage	n/a	a single quality axis (did it push) is at stake
E4 Failure modes anticipated	n/a	no new failure mode beyond a normal git push

No changes needed - imitate this one.

EXCELLENT · score 100 · STRONG One-word continuation, fully resolved by context

chain_step · 2026-08-09

"ok continue"

Why it lands here. A pure continuation acknowledgment that the session fully resolves — proof that a short chain step can be excellent, not just acceptable.

Chain step. Follows 'where are we' in the same session; 'continue' carries the standing task forward with no new ambiguity.

Rubric scorecard

Rubric dimension	Status	Why (evidence)
LAYER 1 - DESIGN (does the prompt drive good behaviour?)		
D1 Clarity & explicitness	Met	resolves cleanly to continuing whatever was just discussed
D2 Specificity & constraints	n/a	no new constraint is introduced by a continuation
D3 Output format & length	n/a	no output format applies
D4 Context & motivation	n/a	n/a for a continuation
D5 Grounding / reference	n/a	n/a
D6 Examples (show-not-tell)	n/a	n/a
D7 Positive framing	n/a	too short to carry a framing signal either way
D8 Uncertainty handling	n/a	n/a
D9 Decomposition fit	Met	single step, not split
D10 Structural economy	Met	exactly as long as it needs to be
LAYER 2 - EVALUABILITY (can success be measured?)		
E1 Success is defined	n/a	a continuation inherits whatever success condition the standing task already had
E2 Criteria are measurable	n/a	n/a
E3 Multidimensional coverage	n/a	n/a
E4 Failure modes anticipated	n/a	n/a

No changes needed - imitate this one.

Good, one fix away

GOOD · score 73 · ADEQUATE Clear question, missing the one anchor

one_shot · 2026-08-09

"where are we"

Why it lands here. A reasonable status check, but it doesn't name which thread or artifact it's asking about.

Rubric scorecard

Rubric dimension	Status	Why (evidence)
LAYER 1 - DESIGN (does the prompt drive good behaviour?)		
D1 Clarity & explicitness	Partial	clear intent (status check) but no named thread/artifact
D2 Specificity & constraints	Partial	no scope given for which work this covers
D3 Output format & length	n/a	no output format needed for a status answer
D4 Context & motivation	n/a	n/a
D5 Grounding / reference	n/a	n/a
D6 Examples (show-not-tell)	n/a	n/a
D7 Positive framing	Met	neutral, non-prohibitive phrasing
D8 Uncertainty handling	n/a	n/a
D9 Decomposition fit	Met	single ask, not split
D10 Structural economy	Met	appropriately brief

Rubric dimension	Status	Why (evidence)
LAYER 2 - EVALUABILITY (can success be measured?)		
E1 Success is defined	n/a	an informational question has no 'correct output' beyond an honest status answer
E2 Criteria are measurable	n/a	n/a
E3 Multidimensional coverage	n/a	n/a
E4 Failure modes anticipated	n/a	n/a

Transformation - turn each gap into a rewrite

Rubric	You wrote	Best-practice rewrite	Principle
D1 Clarity & explicitness	where are we	where are we on the --dry-run flag work for scripts/deploy.sh?	Name the thread/artifact even in a quick status check so the answer can't drift to the wrong topic.

Full rewritten prompt: where are we on the --dry-run flag work for scripts/deploy.sh?

Anti-patterns to kill

BAD · score 65 · WEAK Vague success word after a fully-specified task

chain_step · 2026-08-10

"make it work"

Why it lands here. 'it' resolves fine from context, but 'work' restates no success condition even though the prior turn already had one — leaving it unstated whether that acceptance line still applies.

Chain step. Follows the --dry-run-flag spec in the same session.

Rubric scorecard

Rubric dimension	Status	Why (evidence)
LAYER 1 - DESIGN (does the prompt drive good behaviour?)		
D1 Clarity & explicitness	Met	'it' resolves to the flag work via session context
D2 Specificity & constraints	n/a	no new constraint is introduced this turn
D3 Output format & length	n/a	n/a
D4 Context & motivation	n/a	n/a
D5 Grounding / reference	n/a	n/a
D6 Examples (show-not-tell)	n/a	n/a
D7 Positive framing	n/a	n/a
D8 Uncertainty handling	n/a	n/a
D9 Decomposition fit	Met	single step
D10 Structural economy	Met	appropriately terse in form, though vague in content
LAYER 2 - EVALUABILITY (can success be measured?)		
E1 Success is defined	Missing	'work' states no pass condition of its own, and doesn't confirm the prior acceptance line still applies
E2 Criteria are measurable	Partial	measurable only if 'work' maps onto the earlier acceptance criteria, which is never stated
E3 Multidimensional coverage	n/a	n/a
E4 Failure modes anticipated	n/a	n/a

Transformation - turn each gap into a rewrite

Rubric	You wrote	Best-practice rewrite	Principle
E1 Success is defined	make it work	implement the --dry-run flag now so it passes the acceptance criteria above	A vague success word doesn't inherit a definition from a prior turn automatically — say explicitly that the earlier acceptance criteria still apply.

Full rewritten prompt: implement the --dry-run flag now so it passes the acceptance criteria above

BAD · score 65 · WEAK Floating referent, no message or scope

chain_step · 2026-08-11

"commit this"

Why it lands here. Names the right verb (commit) but 'this' has no prior turn to resolve against, and no message or scope is given.

Rubric scorecard

Rubric dimension	Status	Why (evidence)
LAYER 1 - DESIGN (does the prompt drive good behaviour?)		
D1 Clarity & explicitness	Partial	'this' is the first turn in the session — nothing resolves it
D2 Specificity & constraints	Partial	no commit message or file scope given
D3 Output format & length	n/a	n/a
D4 Context & motivation	n/a	n/a
D5 Grounding / reference	n/a	n/a
D6 Examples (show-not-tell)	n/a	n/a
D7 Positive framing	Met	phrased as what to do
D8 Uncertainty handling	n/a	n/a
D9 Decomposition fit	Met	single step
D10 Structural economy	Met	brief, not over-engineered
LAYER 2 - EVALUABILITY (can success be measured?)		
E1 Success is defined	Partial	a commit existing is clear, but correctness of its message/scope isn't measurable without more
E2 Criteria are measurable	Partial	partially measurable (did a commit happen) but not fully (is it the right one)
E3 Multidimensional coverage	n/a	n/a
E4 Failure modes anticipated	n/a	n/a

Transformation - turn each gap into a rewrite

Rubric	You wrote	Best-practice rewrite	Principle
D1 Clarity & explicitness	commit this	commit the deploy.sh change with a message describing the --dry-run flag	Name what 'this' refers to and give a message — don't leave the target of a destructive-adjacent git op implicit.

Full rewritten prompt: commit the deploy.sh change with a message describing the --dry-run flag

BAD · score 56 · WEAK Category word instead of the actual command

chain_step · 2026-08-11

"version-control it to the shared repo"

Why it lands here. 'version-control' names a category of git operations, not one of them — commit, push, and opening a PR are all consistent with the wording.

Chain step. Follows 'commit this' in the same session.

Rubric scorecard

Rubric dimension	Status	Why (evidence)
LAYER 1 - DESIGN (does the prompt drive good behaviour?)		
D1 Clarity & explicitness	Partial	'version-control' names a category, not a specific git operation
D2 Specificity & constraints	Partial	'to the shared repo' names a target but not which remote/branch
D3 Output format & length	n/a	no meaningful output format beyond the git operation itself
D4 Context & motivation	n/a	motivation wouldn't change which git command applies
D5 Grounding / reference	n/a	no factual grounding needed
D6 Examples (show-not-tell)	n/a	no example needed for a one-line directive
D7 Positive framing	Met	phrased as what to do, not what to avoid
D8 Uncertainty handling	n/a	not a factual-risk task
D9 Decomposition fit	Met	a single step, not artificially split
D10 Structural economy	Met	brief, not over-engineered
LAYER 2 - EVALUABILITY (can success be measured?)		
E1 Success is defined	Missing	no notion of 'done' — a commit-only, a push, and an opened PR are all consistent with the wording
E2 Criteria are measurable	Partial	can't binary-check 'version-controlled' without knowing which git action was meant
E3 Multidimensional coverage	n/a	a single quality axis is at stake here
E4 Failure modes anticipated	n/a	no new failure mode beyond the verb ambiguity already scored

Transformation - turn each gap into a rewrite

Rubric	You wrote	Best-practice rewrite	Principle
D1 Clarity & explicitness	version-control it	commit the deploy.sh change with a message describing the --dry-run flag, then push it to origin/main	Name the actual git command (commit / push / open a PR) — a category word forces the model to guess which one you meant.

Full rewritten prompt: commit the deploy.sh change with a message describing the --dry-run flag, then push it to origin/main

Habits to build

- Replace a vague success word ('work', 'version-control', 'handle it') with the exact action or an explicit acceptance line.
- On the first turn of a git-adjacent ask, name the file/scope and give a message — don't leave a 'this' with nothing to resolve against.
- When a follow-up turn relies on an earlier acceptance line, say so explicitly rather than assuming the model will re-apply it.

Habits you already have

- Appropriately terse chain steps ('now push it', 'ok continue') when the referent genuinely resolves from context — brevity isn't being overused to hide ambiguity.
- When a task is spelled out (the --dry-run flag prompt), it's spelled out completely: scope, format, and a binary acceptance line together.